

Changes in code structure

Procedural code

```
import numpy as np
import math

def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e+6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated!
    return c_D0

def get_wing_cLalpha(AR, phi_LE):
    return 2 + 0 * (AR + phi_LE)

def get_wing_lift(alpha, AR, phi_LE, alpha0):
    clalpha = get_wing_cLalpha(AR, phi_LE)
    return clalpha * (math.radians(alpha) - math.radians(alpha0))

def get_wing_drag(cD0, e, AR, cL):
    return cD0 + cL**2 / (np.pi * AR * e)

def get_wing_polar(A_ref, b, phi_LE, alpha0, e, alphas):
    AR = b**2 / A_ref
    cD0 = estimate_cD0(A_ref, b, phi_LE)

    results = []
    for alpha in alphas:
        cL = get_wing_lift(alpha, AR, phi_LE, alpha0)
        cD = get_wing_drag(cD0, e, AR, cL)
        results.append([cL, cD])

    return np.array(results)

A_ref = 30 # wing reference area
b = 20 # span (m)
phi_LE = 15 # LE sweep angle (°)
alpha0 = -0.6 # zero lift AoA (°)
e = 0.85 # Oswald factor

alpha = 1.5
alphas_polar = np.linspace(0, 3, num=16)

cD0 = estimate_cD0(A_ref, b, phi_LE)
AR = b**2 / A_ref

cL = get_wing_lift(alpha, AR, phi_LE, alpha0)
cD = get_wing_drag(cD0, e, AR, cL)

polar = get_wing_polar(A_ref, b, phi_LE, alpha0, e, alphas_polar)
```

unbound methods can still be used
(Only useful if they also have other
uses outside of the class)

methods that are related directly to the
Object become *instance* methods:
They move inside the class, can be
accessed as attributes of an object
(the *instance* of the class)

Variables specifying object-specific
values are provided once at object
creation and can then be processed and
stored as attributes by the `__init__()`
method. There's no need for the outer
scope to store them.

Intermediate results that
don't change can be
computed during initialisation
and stored in the object

OOP code

```
import numpy as np
import math

def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e+6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated!
    return c_D0

class Wing:
    def __init__(self, A_ref, b, phi_LE=0, alpha0=0, e=0.85):
        self.A_ref = A_ref
        self.b = b
        self.phi_LE = phi_LE
        self.alpha0 = alpha0
        self.e = e

        self.AR = b**2 / A_ref
        self.cD0 = estimate_cD0(A_ref, b, phi_LE)
        self.clalpha = 2 + 0 * (self.AR + self.phi_LE)

    def get_lift(self, alpha):
        return self.clalpha * (math.radians(alpha) -
                               math.radians(self.alpha0))

    def get_drag(self, cL):
        return self.cD0 + cL**2 / (np.pi * self.AR * self.e)

    def get_polar(self, alphas) → np.array:
        results = []
        for alpha in alphas:
            cL = self.get_lift(alpha)
            cD = self.get_drag(cL)
            results.append([cL, cD])

        return np.array(results)

thiswing = Wing(A_ref=30, b=20, phi_LE=15, alpha0=-0.6, e=0.85)

alpha = 1.5
alphas_polar = np.linspace(0, 3, num=16)

cL = thiswing.get_wing_lift(alpha)
cD = thiswing.get_wing_drag(cL)

polar = thiswing.get_polar(alphas_polar)
```

Procedural vs. Object-oriented Code – Cheat Sheet

Differences in data handling

Procedural code

```
import numpy as np
import math

def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e+6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated!
    return c_D0

def get_wing_cLalpha(AR, phi_LE):
    return 2 + 0 * (AR + phi_LE)

def get_wing_lift(alpha, AR, phi_LE, alpha0):
    clalpha = get_wing_cLalpha(AR, phi_LE)
    return clalpha * (math.radians(alpha) - math.radians(alpha0))

def get_wing_drag(cD0, e, AR, cL):
    return cD0 + cL**2 / (np.pi * AR * e)

def get_wing_polar(A_ref, b, phi_LE, alpha0, e, alphas):
    AR = b**2 / A_ref
    cD0 = estimate_cD0(A_ref, b, phi_LE)

    results = []
    for alpha in alphas:
        cL = get_wing_lift(alpha, AR, phi_LE, alpha0)
        cD = get_wing_drag(cD0, e, AR, cL)
        results.append([cL, cD])

    return np.array(results)
```

`A_ref = 30` # wing reference area
`b = 20` # span (m)
`phi_LE = 15` # LE sweep angle (°)
`alpha0 = -0.6` # zero lift AoA (°)
`e = 0.85` # Oswald factor

`alpha = 1.5`
`alphas_polar = np.linspace(0, 3, num=16)`

`cD0 = estimate_cD0(A_ref, b, phi_LE)`
`AR = b**2 / A_ref`

`cL = get_wing_lift(alpha, AR, phi_LE, alpha0)`
`cD = get_wing_drag(cD0, e, AR, cL)`

`polar = get_wing_polar(A_ref, b, phi_LE, alpha0, e, alphas_polar)`

methods are “unbound”,
require all relevant data
as call arguments

`__init__()` is called
implicitly at instantiation
and can calculate and
store whatever attributes
are necessary for the
object to work, as
`self.<attr>`

Instance methods
implicitly receive `self` as
first argument and can
access all instance
attributes

Object-specific data is
defined and stored in
the “outer” scope...

Object-specific data is
provided once during
instantiation and stored
in the resulting object

This object contains the
data and the correct
methods needed for the
functionality.

Outer scope needs to
“know” which methods
are appropriate

Methods names are
verbs because
methods *do* something

OOP code

```
import numpy as np
import math

def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e+6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated!
    return c_D0

class Wing:
    def __init__(self, A_ref, b, phi_LE=0, alpha0=0, e=0.85):
        self.A_ref = A_ref
        self.b = b
        self.phi_LE = phi_LE
        self.alpha0 = alpha0
        self.e = e

        self.AR = b**2 / A_ref
        self.cD0 = estimate_cD0(A_ref, b, phi_LE)
        self.cLalpha = 2 + 0 * (self.AR + self.phi_LE)

    def get_lift(self, alpha):
        return self.cLalpha \
            * (math.radians(alpha) - math.radians(self.alpha0))

    def get_drag(self, cL):
        return self.cD0 + cL**2 / (np.pi * self.AR * self.e)

    def get_polar(self, alphas) → np.array:
        results = []
        for alpha in alphas:
            cL = self.get_lift(alpha)
            cD = self.get_drag(cL)
            results.append([cL, cD])

        return np.array(results)
```

`testwing = Wing(A_ref=30, b=20, phi_LE=15, alpha0=-0.6, e=0.85)`

`alpha = 1.5`
`alphas_polar = np.linspace(0, 3, num=16)`

`cL = testwing.get_lift(alpha)`
`cD = testwing.get_drag(cL)`

`polar = testwing.get_polar(alphas_polar)`

All methods defined within
the class are available as
instance attributes

The class name and the name of the object are
nouns because they describe objects

Methods names inside in the class are still
verbs because they specify what an object of
this class *does*.

Procedural code structure – cheat sheet

Information flow

“Outer” scope (use-case specific)

```
import numpy as np
import math

[...]
```

Object-specific data

```
A_ref = 30 # wing reference area
b = 20 # span (m)
phi_LE = 15 # LE sweep angle (°)
alpha0 = -0.6 # zero lift AoA (°)
e = 0.85 # Oswald factor
```

Analysis-specific data

```
alpha = 1.5
alphas_polar = np.linspace(0, 3, num=16)
```

```
cD0 = estimate_cD0(A_ref, b, phi_LE)
AR = b**2 / A_ref
```

```
cL = get_wing_lift(AR, phi_LE, alpha0, alpha)
cD = get_wing_drag(cD0, e, AR, cL)
```

```
polar = get_wing_polar(A_ref, b, phi_LE, alpha0, e, alphas_polar)
```

The outer scope provides the input data, every time it's used.

We can change data as we like but we also must take care to keep it consistent, including intermediate results.

“Inner” scope (Core functionality)

```
def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e6)**(1/5) # accurately calibrated!
    return c_D0
```

```
def get_wing_cLalpha(AR, phi_LE):
    return 2 + 0 * (AR + phi_LE)
```

```
def get_wing_lift(alpha, AR, phi_LE, alpha0):
    c_lalpha = get_wing_cLalpha(AR, phi_LE)
    return c_lalpha * (math.radians(alpha) - math.radians(alpha0))
```

```
def get_wing_drag(cD0, e, AR, cL):
    return cD0 + cL**2 / (np.pi * AR * e)
```

```
def get_wing_polar(A_ref, b, phi_LE, alpha0, e, alphas):
    AR = b**2 / A_ref
    cD0 = estimate_cD0(A_ref, b, phi_LE)
```

```
    results = []
    for alpha in alphas:
        cL = get_wing_lift(alpha, AR, phi_LE, alpha0)
        cD = get_wing_drag(cD0, e, AR, cL)
        results.append([cL, cD])

    return np.array(results)
```

The “inner” scope has no memory
stores no intermediate results
Relies on user to provide correct inputs
for every calculation

Methods are called
(and may return
something)

Object-Oriented code structure – cheat sheet

Information flow

Outer scope

```
import numpy as np
```

```
[...]
```

```
testwing = Wing(A_ref=30, b=20, phi_LE=15, alpha0=-0.6, e=0.85)
```

```
alpha = 1.5  
alphas_polar = np.linspace(0, 3, num=16)
```

```
cL = testwing.get_lift(alpha)  
cD = testwing.get_drag(cL)
```

```
polar = testwing.get_polar(alphas_polar)
```

```
print(testwing.clalpha)  
# object-internal data remains available!
```

The outer scope provides the input data, once, and the object “knows” what to do with it.

The object can be passed as an argument to other methods/classes, or even saved to disk and loaded later.

If we want to change inputs, it's safer to create a new object:

```
testwing.A_ref = 10 # This is a terrible idea!  
# ...unless you write some extra code to make sure that  
# AR, cD0 and clalpha are updated afterwards...  
  
# better:  
testwing2 = Wing(A_ref=10, b=20, phi_LE=15, alpha0=-0.6, e=0.85)  
# Now we can keep both wings around, and each wing contains its own data
```

The object returned includes all methods and other attributes defined in the class, including those defined by `__init__()`

Classes are **instantiated**, not called!
This produces an *object* which is an *instance* of the class

Object-specific data

Instance methods can be *called*, just like all other methods

Inner scope

```
import numpy as np  
import math
```

```
def estimate_cD0(A_ref: float, b: float, phi_LE: float) -> float:  
    c_ref = A_ref / b # mean chord length  
    Re = 1e+6 * c_ref # representative Reynolds number  
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated estimate!  
    return c_D0
```

```
class Wing:
```

```
def __init__(self, A_ref, b, phi_LE=0, alpha0=0, e=0.85):  
    self.A_ref = A_ref  
    self.b = b  
    self.phi_LE = phi_LE  
    self.alpha0 = alpha0  
    self.e = e
```

```
self.AR = b**2 / A_ref  
self.cD0 = estimate_cD0(A_ref, b, phi_LE)  
self.clalpha = 2 + 0 * (self.AR + self.phi_LE)
```

```
def get_lift(self, alpha):  
    return self.clalpha * (math.radians(alpha) -  
                           math.radians(self.alpha0))
```

```
def get_drag(self, cL):  
    return self.cD0 + cL**2 / (np.pi * self.AR * self.e)
```

```
def get_polar(self, alphas) -> np.array:  
    results = []  
    for alpha in alphas:  
        cL = self.get_lift(alpha)  
        cD = self.get_drag(cL)  
        results.append([cL, cD])  
  
    return np.array(results)
```

<builtin `__new__()`>
creates a new object
adds all methods from the class
calls its `__init__()` method
returns the initialized object

All methods **inside** the class can access all *attributes* and *methods* as `self.<name>`

Direct inputs *and* intermediate results are stored and are available to all internal methods